

CSCI 5540 – Fall 26

Tutorial 2

DENG Yangtao

Credits to CMU Machine Learning Systems Lecture 4

Overview

1

Why GPUs?

Massive data parallelism

2

How to program them?

Grid, block, thread, kernel

3

What makes code fast?

Coherent control and data reuse

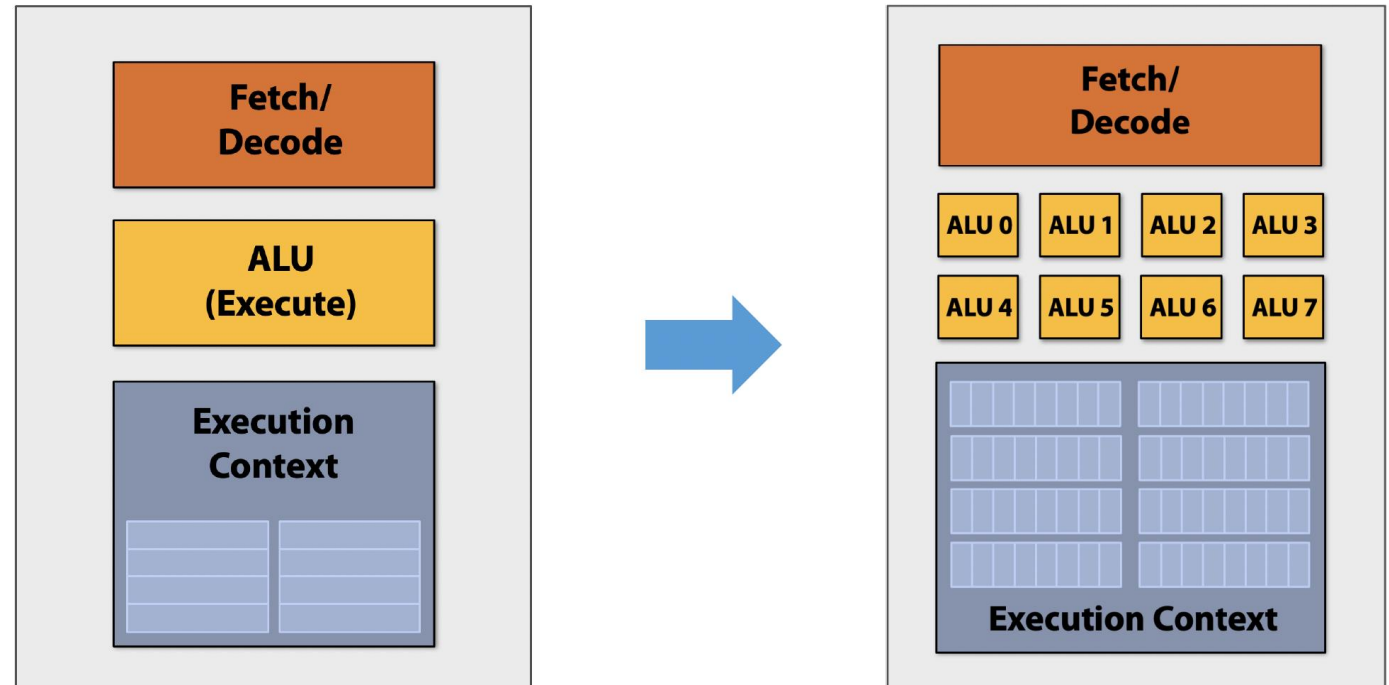
Why GPUs? Single Instruction, Multiple Data (SIMD)

CPU

- A few sophisticated cores
- Optimized for low latency
- Good at irregular control flow

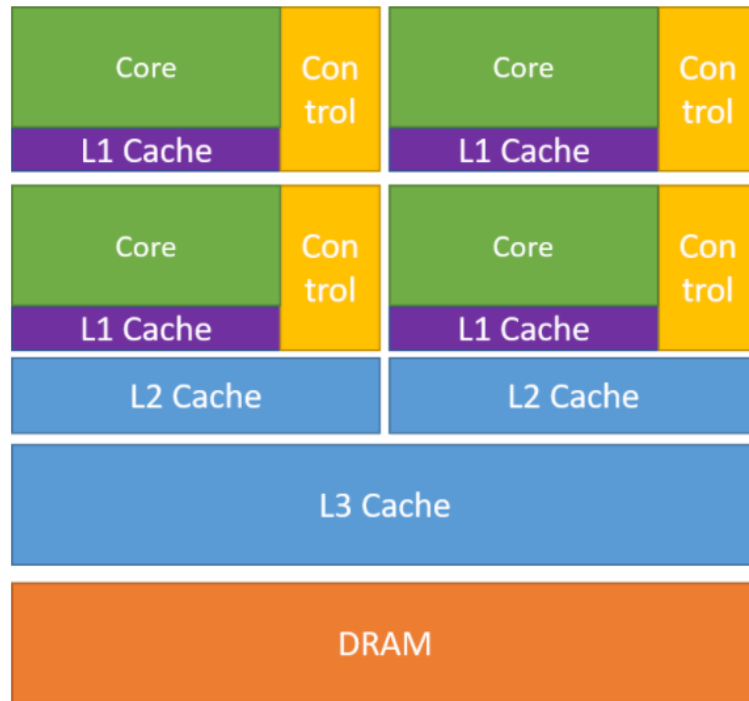
GPU

- Many simpler execution lanes
- Optimized for total throughput
- Best for regular data-parallel work



The GPU wins by keeping **many operations** in flight.

Massive Parallel Computing Units



CPU



GPU

Massive Streaming Multiprocessors (SM) with Tensor Cores

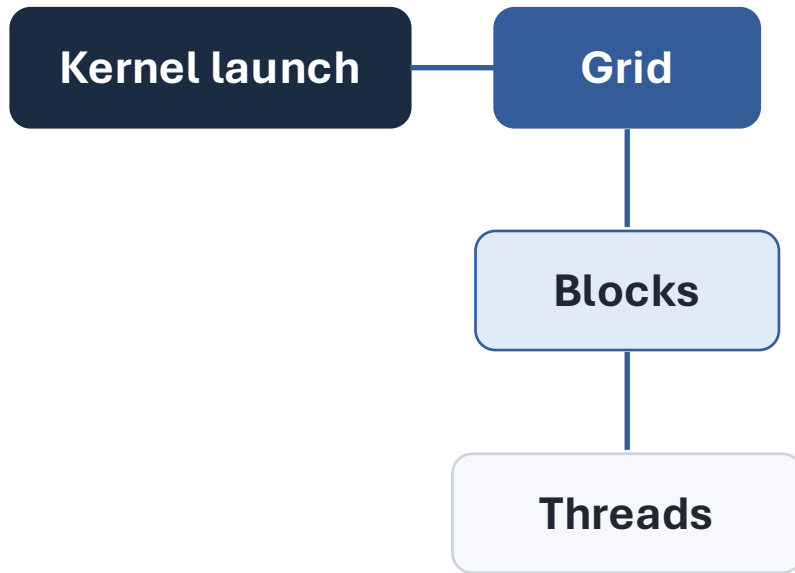
CUDA exposes the GPU's parallel structure

CUDA

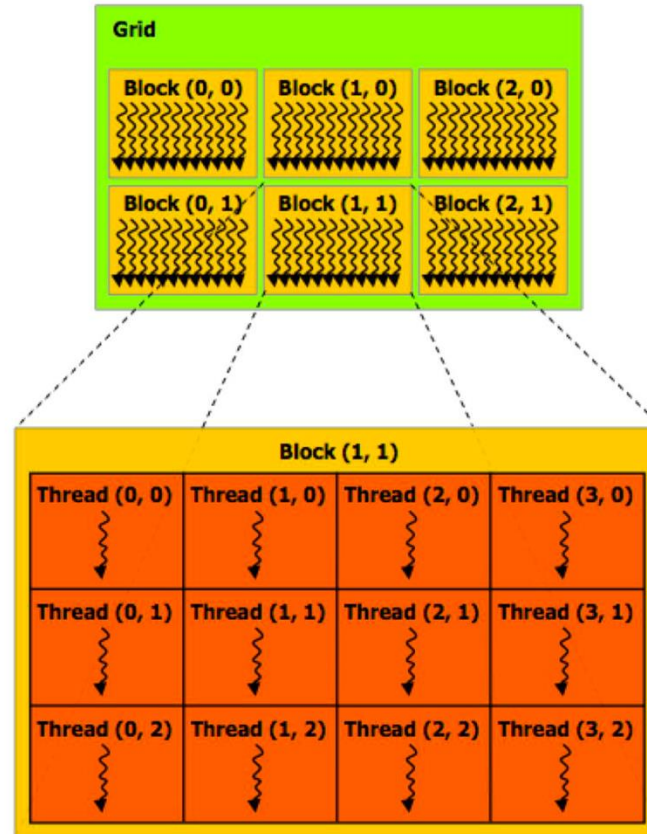
- C/C++-like programming model
- Introduced with NVIDIA Tesla in 2007
- Makes parallelism and memory explicit

**Low abstraction distance:
software concepts closely match hardware behavior.**

Grid, block, and thread



- Thread: one logical execution instance
- Block: threads that can cooperate
- Grid: all blocks in one kernel launch



```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x,
               Ny/threadsPerBlock.y, 1);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will trigger execution of 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

Run on
CPU

Example: 6 blocks × 12 threads = 72 threads

A CUDA kernel: one thread, one output

Serial execution: running as part of normal C/C++ application on CPU

Host

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x,
               Ny/threadsPerBlock.y, 1);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will trigger execution of 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

Bulk launch of many CUDA threads
“launch a grid of CUDA thread blocks”
Call returns when all threads have terminated



Device

```
// kernel definition
__global__ void matrixAdd(float A[Ny][Nx],
                          float B[Ny][Nx],
                          float C[Ny][Nx])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    C[j][i] = A[j][i] + B[j][i];
}
```

`__global__` denotes a CUDA kernel function runs on GPU

Each thread computes its overall grid thread id from its position in its block (`threadIdx`) and its block's position in the grid (`blockIdx`)

CUDA kernel: executed in parallel on multiple CUDA cores

Host code launches device code



**Kernel launches are normally asynchronous:
the CPU can continue after submission.**

Host memory, device memory, and CUDA APIs

Host memory

Device memory

cudaMalloc
cudaMemcpy
kernel launch

Host-side APIs: called by CPU code
Device-side APIs: called inside kernels

```
float* A = new float[N];

// populate host address space pointer A
for (int i=0 i<N; i++)
    A[i] = (float)i;

int bytes = sizeof(float) * N
float* deviceA;          // allocate buffer in
cudaMalloc(&deviceA, bytes); // device address space

// populate deviceA
cudaMemcpy(deviceA, A, bytes, cudaMemcpyHostToDevice);

// note: deviceA[i] is an invalid operation here (cannot
- // manipulate contents of deviceA directly from host.
// Only from device code.)
```

Minimize CPU↔GPU transfers; keep intermediates on the device.

Thread count is explicit: check the boundary

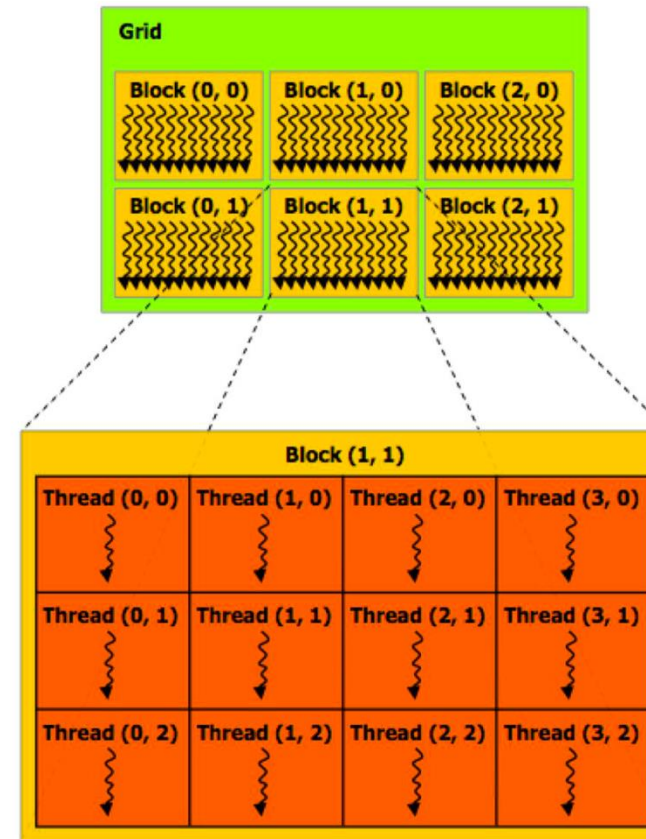
N = 1000 elements

threadsPerBlock = 256

numBlocks = $\text{ceil}(1000 / 256) = 4$

$4 \times 256 = 1024$ threads
24 threads have no valid element

if ($i < N$) $C[i] = A[i] + B[i];$



- Number of kernel invocations is not determined by size of data collection

Warp divergence: when threads disagree

Warp = 32 threads scheduled together

if path



else path

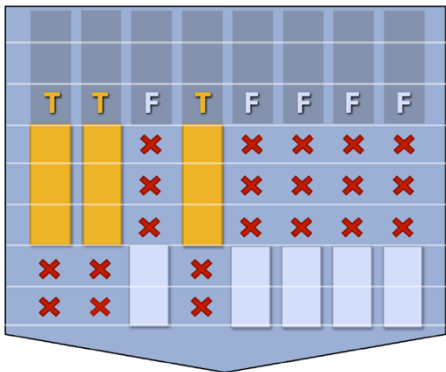


Both paths execute serially;
non-participating lanes are masked.

Time (clocks)



1 2 ... 8
ALU 1 ALU 2 ALU 8



Not all ALUs do useful work!
Worst case: 1/8 peak performance

```
// kernel definition
__global__ void f(float A[N])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    float x = A[i];
    if (x > 0) {
        x = 2.0f * x;
    } else {
        x = exp(x, 5.0f);
    }
    A[i] = x;
}
```

Avoid divergence inside a warp - not every if statement.

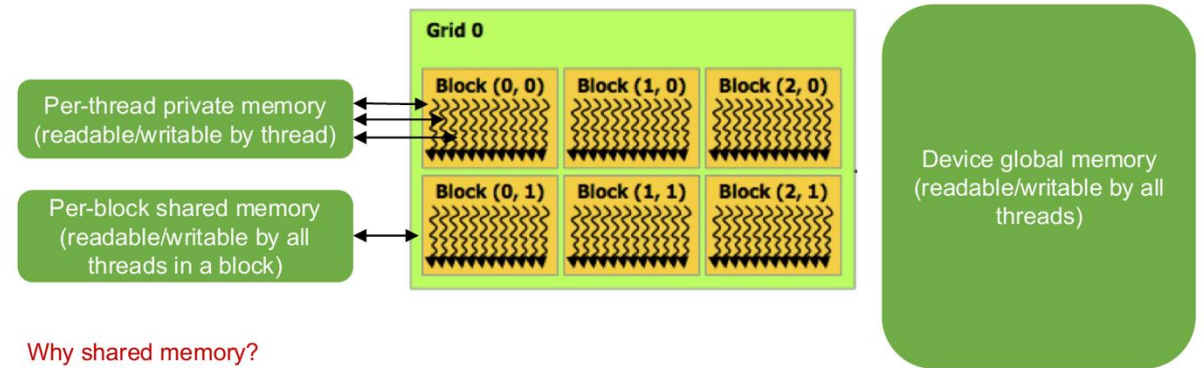
CUDA device memory model

Registers one thread **fastest / tiny**

Shared memory one block **fast / explicit**

Global memory all threads **large / high latency**

Global memory is logical;
HBM/GDDR is the physical DRAM.

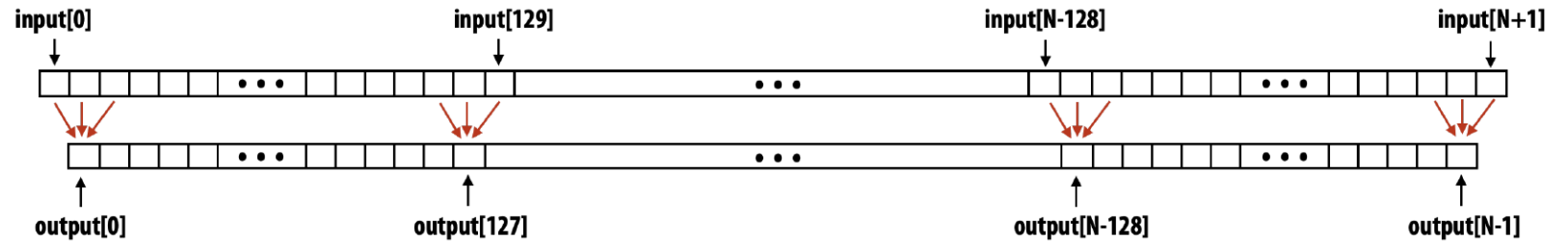


Why shared memory?
Enable cooperation across threads in a block

Shared memory enables data reuse and cooperation within one block.

Example: repeated loads in 1D convolution

Neighboring outputs repeatedly read the same inputs.



Each output uses three inputs

$$\text{output}[i] = (\text{input}[i] + \text{input}[i+1] + \text{input}[i+2]) / 3$$

$$128 \text{ outputs} \times 3 \text{ loads} = 384 \text{ global-memory loads}$$

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {

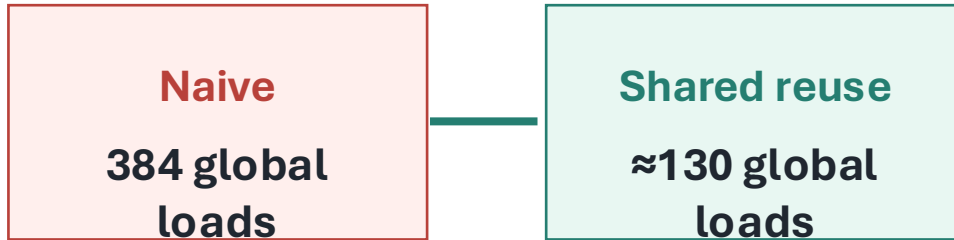
    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local variable

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += input[index + i];

    output[index] = result / 3.f;
}
```

each thread computes result for one element

Reuse overlapping inputs in shared memory



- Threads cooperatively load the support region
- One global load can serve several outputs
- All threads wait before consuming shared data

__syncthreads();

```
#define THREADS_PER_BLK 128

__global__ void convolve(int N, float* input, float* output) {

    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local variable

    __shared__ float support[THREADS_PER_BLK*2]; // per-block allocation
    support[threadIdx.x] = input[index];
    if (threadIdx.x < 2) {
        support[THREADS_PER_BLK + threadIdx.x] = input[index+THREADS_PER_BLK];
    }

    __syncthreads();

    float result = 0.0f; // thread-local variable
    for (int i=0; i<3; i++)
        result += support[threadIdx.x + i];

    output[index] = result / 3.f;
}
```

All threads cooperatively load block's support region from global into shared memory (total of 130 loads instead of 3 * 128 loads)

barrier (all threads in block)

each thread computes result for one element

Synchronization and the basic CUDA checklist

1

Expose parallel work one clear task per thread

2

Keep warps coherent avoid warp-internal divergence

3

Reduce global loads reuse data in shared memory

4

Synchronize correctly before consuming shared results

GPU code becomes fast when parallel work stays useful.

Reminders

- Student seminar
 - Presenter: present the assigned required papers.
 - Reviewers: ask questions and write a summary (**link will be updated for submission**).
 - Rest of the class: submit one insightful question **before each class** (**link will be updated for submission**).
- Group project
 - Submit a 2-page PDF draft proposal (including references) by **September 30** (**link will be updated for submission**).

Thanks!